

Notebook 1: Setting Up Your Local AI Environment for Biology

Introduction

Welcome to the first notebook of the "Local AI for Biology" workshop! This notebook will guide you through setting up a complete local environment for running Large Language Models (LLMs) and AI tools on your own computer or server, without relying on cloud services.

Why Local AI?

Running AI models locally offers several critical advantages, especially in biology and biomedical research:

1. **Data Privacy:** Your sensitive patient data, proprietary research, or genetic information never leaves your control
2. **Compliance:** Easier to meet HIPAA, GDPR, and other regulatory requirements
3. **Cost Predictability:** No surprise cloud bills from token usage
4. **Offline Access:** Work without internet connectivity
5. **Customization:** Full control over model selection and fine-tuning
6. **Speed:** Reduced network latency for real-time applications

What You'll Learn

By the end of this notebook, you will:

- Understand the hardware requirements for local LLM deployment
- Install and configure Conda/Mamba for environment management
- Set up Docker for containerized AI applications
- Configure GPU acceleration (CUDA) for faster inference
- Verify your installation and run basic tests
- Understand the trade-offs between different deployment options

Part 1: Understanding Hardware Requirements

Before we begin installation, let's understand what hardware you need for running LLMs locally.

1.1 GPU Requirements

What is a GPU? A Graphics Processing Unit (GPU) is a specialized processor originally designed for rendering graphics, but now widely used for parallel computations in AI. GPUs can perform thousands of operations simultaneously, making them much faster than CPUs for matrix operations common in neural networks.

Why GPUs Matter for LLMs

- LLMs consist of billions of parameters (weights)
- Each prediction requires massive matrix multiplications

- GPUs can perform these operations 10-100x faster than CPUs

Recommended GPUs for Local AI:

GPU Model	VRAM	Best For	Approximate Cost
NVIDIA RTX 3060	12GB	Small models (7B params)	\$300-400
NVIDIA RTX 3090	24GB	Medium models (13B params)	\$1,000-1,500
NVIDIA RTX 4090	24GB	Medium-large models	\$1,600-2,000
NVIDIA A100	40-80GB	Large models (70B params)	\$10,000+
NVIDIA H100	80GB	Very large models	\$30,000+

VRAM (Video RAM) Explained: VRAM is the memory on your GPU. LLM size requirements:

- 7B parameter model (quantized): ~4-6GB VRAM
- 13B parameter model (quantized): ~8-10GB VRAM
- 70B parameter model (quantized): ~35-40GB VRAM

Quantization: A technique to compress models by reducing numerical precision (e.g., from 32-bit to 8-bit or 4-bit), significantly reducing memory requirements with minimal accuracy loss.

1.2 CPU and RAM Requirements

Even with a GPU, you need adequate CPU and system RAM:

- **CPU:** Modern multi-core processor (Intel i5/i7, AMD Ryzen 5/7 or better)
- **RAM:** 16GB minimum, 32GB+ recommended
- **Storage:** SSD with 100GB+ free space (models can be 5-50GB each)

1.3 Checking Your System

Let's check what you currently have:

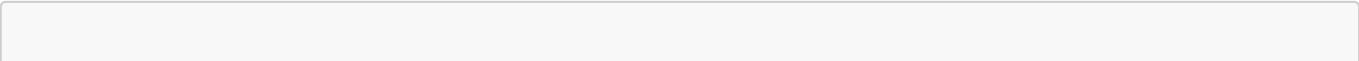
```
# Check GPU (NVIDIA)
nvidia-smi

# Check CPU
lscpu | grep "Model name"

# Check RAM
free -h

# Check disk space
df -h
```

Expected Output from `nvidia-smi`:



```
+-----+
---+
| NVIDIA-SMI 535.104.05      Driver Version: 535.104.05      CUDA Version: 12.2
|
|-----+-----+-----+
---+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr.
ECC |
| Fan   Temp   Perf   Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute
M. |
|=====+=====+=====+
===|
|    0   NVIDIA RTX 3090        Off    | 00000000:01:00.0 Off |
N/A |
| 30%    45C    P8      25W / 350W |      0MiB / 24576MiB |      0%
Default |
+-----+-----+-----+
---+
```

Interpretation:

- **Driver Version:** NVIDIA driver version (535.104.05 here)
- **CUDA Version:** Compatible CUDA version (12.2)
- **GPU Name:** Your GPU model (RTX 3090 with 24GB VRAM)
- **Memory-Usage:** Current VRAM usage (0MiB used / 24576MiB total)
- **GPU-Util:** Current GPU utilization (0% = idle)
- **Temp:** GPU temperature (45°C = cool)

Part 2: Installing Conda/Mamba

What is Conda? Conda is a package manager and environment manager. It allows you to:

- Create isolated Python environments
- Install packages without conflicts
- Reproduce environments across machines
- Manage dependencies automatically

What is Mamba? Mamba is a faster, drop-in replacement for Conda. It uses parallel processing and a better dependency solver, making installations 5-10x faster.

2.1 Installing Mambaforge (Recommended)

Mambaforge combines Conda with Mamba and uses the community-driven conda-forge repository.

Step 1: Download Mambaforge

```
# Linux/WSL
wget https://github.com/conda-
forge/miniforge/releases/latest/download/Mambaforge-Linux-x86_64.sh
```

```
bash Mambaforge-Linux-x86_64.sh

# macOS
wget https://github.com/conda-
forge/miniforge/releases/latest/download/Mambaforge-MacOSX-x86_64.sh
bash Mambaforge-MacOSX-x86_64.sh
```

Step 2: Follow Installation Prompts

- Accept the license
- Confirm installation location (default: `/home/username/mambaforge`)
- Allow initialization (adds conda to your shell)

Step 3: Restart Shell

```
source ~/.bashrc # Linux
# or
source ~/.zshrc  # macOS
```

Step 4: Verify Installation

```
mamba --version
# Expected output: mamba 1.5.3, conda 23.9.0
```

2.2 Creating Your First Environment

Environments prevent package conflicts by isolating dependencies.

```
# Create an environment named 'local-ai' with Python 3.10
mamba create -n local-ai python=3.10 -y

# Activate the environment
mamba activate local-ai

# Verify you're in the environment
which python
# Should show: /home/username/mambaforge/envs/local-ai/bin/python
```

Understanding the Output:

- `-n local-ai`: Names the environment "local-ai"
- `python=3.10`: Installs Python 3.10
- `-y`: Automatically confirms installation
- `mamba activate`: Switches to the environment

2.3 Installing Basic Dependencies

```
# Install common AI/ML packages
mamba install numpy pandas scipy matplotlib seaborn jupyter ipykernel -y

# Install bioinformatics tools
mamba install -c bioconda biopython pysam -y

# Verify installations
python -c "import numpy; print(f'NumPy version: {numpy.__version__}')"
python -c "import Bio; print(f'Biopython version: {Bio.__version__}')
```

Package Explanations:

- **numpy**: Numerical computing library for arrays and matrices
 - **pandas**: Data manipulation and analysis (DataFrames)
 - **scipy**: Scientific computing (statistics, optimization)
 - **matplotlib/seaborn**: Data visualization
 - **jupyter**: Interactive notebooks (like this one!)
 - **biopython**: Tools for biological computation (sequence analysis, file parsing)
 - **pysam**: Python interface to SAM/BAM genomic files
-

Part 3: Installing Docker

What is Docker? Docker is a platform for running applications in isolated containers. Containers are like lightweight virtual machines that:

- Package the application with all dependencies
- Run consistently across different systems
- Isolate applications from each other
- Start/stop quickly

Why Docker for AI?

- Pre-built containers with all LLM dependencies
- Avoid "dependency hell" (conflicting library versions)
- Easy to share and reproduce setups
- GPU access from containers

3.1 Installing Docker

Ubuntu/Debian Linux:

```
# Update package index
sudo apt update

# Install prerequisites
```

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common -y

# Add Docker's official GPG key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io -y

# Add your user to docker group (avoid needing sudo)
sudo usermod -aG docker $USER

# Log out and back in for group changes to take effect
```

macOS:

1. Download Docker Desktop from <https://www.docker.com/products/docker-desktop/>
2. Install the .dmg file
3. Open Docker Desktop and follow setup wizard

Windows:

1. Install WSL2 (Windows Subsystem for Linux)
2. Download Docker Desktop for Windows
3. Enable WSL2 integration in Docker settings

3.2 Verifying Docker Installation

```
# Check Docker version
docker --version
# Expected: Docker version 24.0.7, build afdd53b

# Test Docker with hello-world
docker run hello-world
```

Expected Output:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

3.3 Installing NVIDIA Container Toolkit (GPU Support)

To use your GPU inside Docker containers, you need the NVIDIA Container Toolkit.

```
# Add NVIDIA package repositories
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list

# Install nvidia-docker2
sudo apt update
sudo apt install nvidia-docker2 -y

# Restart Docker daemon
sudo systemctl restart docker

# Test GPU access in Docker
docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi
```

Understanding the Test Command:

- `docker run`: Run a container
- `--rm`: Remove container after it exits
- `--gpus all`: Grant access to all GPUs
- `nvidia/cuda:12.2.0-base-ubuntu22.04`: Pre-built image with CUDA
- `nvidia-smi`: Command to run inside container

If you see the `nvidia-smi` output with your GPU, Docker GPU access works!

Part 4: Installing CUDA Toolkit (For Native Python)

If you want to run LLMs directly with Python (not in Docker), you need CUDA installed on your system.

What is CUDA? CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform. It allows programs to use the GPU for general-purpose processing, not just graphics.

4.1 Checking CUDA Compatibility

```
# Check your NVIDIA driver
nvidia-smi | grep "Driver Version"

# Check compatible CUDA versions
# NVIDIA Driver 535+ supports CUDA 12.2
```

Driver-CUDA Compatibility Table:

Driver Version	CUDA Version
450.x+	11.0
470.x+	11.4
515.x+	11.7
525.x+	12.0
535.x+	12.2

4.2 Installing CUDA Toolkit

Ubuntu/Debian:

```
# Download CUDA 12.2 installer
wget
https://developer.download.nvidia.com/compute/cuda/12.2.0/local_installers/
cuda_12.2.0_535.54.03_linux.run

# Run installer (deselect driver if already installed)
sudo sh cuda_12.2.0_535.54.03_linux.run

# Add CUDA to PATH
echo 'export PATH=/usr/local/cuda-12.2/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.2/lib64:$LD_LIBRARY_PATH'
>> ~/.bashrc
source ~/.bashrc

# Verify installation
nvcc --version
```

Expected Output:

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Tue_Jun_13_19:16:58_PDT_2023
Cuda compilation tools, release 12.2, V12.2.91
```

4.3 Installing PyTorch with CUDA

PyTorch is the most popular deep learning framework. Let's install it with CUDA support:

```
# Activate your environment
mamba activate local-ai

# Install PyTorch with CUDA 12.1 support
pip install torch torchvision torchaudio --index-url
```



```
https://download.pytorch.org/whl/cu121

# Verify installation
python -c "import torch; print(f'PyTorch version: {torch.__version__}');
print(f'CUDA available: {torch.cuda.is_available()}'); print(f'CUDA
version: {torch.version.cuda}')"

```

Expected Output:

```
PyTorch version: 2.1.0+cu121
CUDA available: True
CUDA version: 12.1

```

If `CUDA available: False`, troubleshoot:

1. Check NVIDIA driver: `nvidia-smi`
2. Verify CUDA installation: `nvcc --version`
3. Ensure PyTorch CUDA version matches installed CUDA

Part 5: Installing LLM Frameworks

Now that we have the foundation, let's install frameworks for running LLMs locally.

5.1 Installing llama.cpp

What is llama.cpp? llama.cpp is a C++ implementation of LLaMA and other LLMs, optimized for:

- Low memory usage (quantization support)
- CPU inference (no GPU required, but GPU optional)
- Fast inference
- Minimal dependencies

```
# Clone the repository
cd ~
git clone https://github.com/ggerranov/llama.cpp.git
cd llama.cpp

# Build with CUDA support (if you have CUDA)
make LLAMA_CUDA=1

# Or build CPU-only version
make

# Test build
./main --version

```

Building on Windows: Download w64devkit from <https://github.com/skeeto/w64devkit/releases> Extract and run w64devkit.exe Navigate to llama.cpp directory Run `make`

5.2 Installing Ollama

What is Ollama? Ollama is a tool that makes running LLMs as easy as running `docker run`. It handles:

- Model downloads
- Model management
- API server
- GPU acceleration

```
# Linux/macOS
curl -fsSL https://ollama.com/install.sh | sh

# Verify installation
ollama --version

# Download and run Llama 2 7B model
ollama run llama2

# This downloads the model (~4GB) and starts a chat interface
```

Using Ollama:

```
# List available models
ollama list

# Run a specific model
ollama run llama2

# Run with custom parameters
ollama run llama2 --temperature 0.7 --top-k 40

# Pull a model without running
ollama pull mistral

# Remove a model
ollama rm llama2
```

5.3 Installing Hugging Face Transformers

What is Hugging Face? Hugging Face is a company and platform providing:

- Pre-trained models (100,000+ models)
- Transformers library (state-of-the-art NLP)
- Datasets library
- Model hosting and sharing

```
# Activate environment
mamba activate local-ai

# Install transformers and dependencies
pip install transformers accelerate sentencepiece protobuf

# Install datasets library
pip install datasets

# Test installation
python -c "import transformers; print(f'Transformers version:
{transformers.__version__}')"

```

Your First Model with Transformers:

```
from transformers import pipeline

# Load a pre-trained text generation model
generator = pipeline('text-generation', model='gpt2')

# Generate text
result = generator("Bioinformatics is", max_length=50,
num_return_sequences=1)
print(result[0]['generated_text'])

```

Part 6: Testing Your Setup

Let's verify everything works with comprehensive tests.

6.1 GPU Test

```
import torch

# Check PyTorch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"CUDA version: {torch.version.cuda}")
    print(f"Number of GPUs: {torch.cuda.device_count()}")
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory /
1e9:.2f} GB")

# Perform a simple GPU computation
x = torch.randn(1000, 1000, device='cuda')
y = torch.randn(1000, 1000, device='cuda')

```

```
z = torch.matmul(x, y)
print("GPU computation successful!")
else:
    print("CUDA not available. CPU-only mode.")
```

6.2 Transformers Test

```
from transformers import AutoTokenizer, AutoModel
import torch

# Load a small model (BERT-base)
print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

print("Loading model...")
model = AutoModel.from_pretrained("bert-base-uncased")

# Move to GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

# Test tokenization and inference
text = "Bioinformatics combines biology, computer science, and statistics."
inputs = tokenizer(text, return_tensors="pt").to(device)

print(f"\nInput text: {text}")
print(f"\nTokenized inputs: {inputs}")

# Generate embeddings
with torch.no_grad():
    outputs = model(**inputs)

print(f"\nOutput shape: {outputs.last_hidden_state.shape}")
print(f"Output device: {outputs.last_hidden_state.device}")
print("\nSuccess! Model loaded and ran inference.")
```

6.3 Ollama Test

```
# Test if Ollama service is running
ollama list

# If not running, start it
ollama serve &

# Pull a small model
ollama pull tinyllama

# Test chat
ollama run tinyllama "Explain DNA in one sentence"
```

6.4 llama.cpp Test

```
# Navigate to llama.cpp directory
cd ~/llama.cpp

# Download a small GGUF model
wget https://huggingface.co/TheBloke/TinyLlama-1.1B-Chat-v1.0-
GGUF/resolve/main/tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf

# Run inference
./main -m tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf -p "What is bioinformatics?"
-n 128
```

Understanding the Command:

- `./main`: Run the llama.cpp executable
- `-m`: Specify model file
- `-p`: Prompt text
- `-n 128`: Generate 128 tokens

Part 7: Environment Management Best Practices

7.1 Creating Project-Specific Environments

```
# Create environment for a specific project
mamba create -n genomics-llm python=3.10 biopython pandas -y

# Export environment (for sharing)
mamba activate genomics-llm
mamba env export > environment.yml

# Recreate environment on another machine
mamba env create -f environment.yml
```

7.2 Using requirements.txt

```
# Generate requirements file
pip freeze > requirements.txt

# Install from requirements
pip install -r requirements.txt
```

7.3 Docker for Reproducibility

Create a **Dockerfile**:

```
# Use NVIDIA CUDA base image
FROM nvidia/cuda:12.2.0-runtime-ubuntu22.04

# Install Python and dependencies
RUN apt-get update && apt-get install -y \\\
    python3.10 \\\
    python3-pip \\\
    git

# Install Python packages
RUN pip3 install torch transformers accelerate

# Set working directory
WORKDIR /workspace

# Copy your code
COPY . /workspace

# Run your application
CMD ["python3", "your_script.py"]
```

Build and run:

```
# Build image
docker build -t bio-llm:latest .

# Run with GPU
docker run --gpus all -it bio-llm:latest
```

Part 8: Troubleshooting Common Issues

8.1 CUDA Out of Memory

Problem: `RuntimeError: CUDA out of memory`

Solutions:

1. Use smaller models or quantized versions
2. Reduce batch size
3. Use gradient checkpointing
4. Clear cache: `torch.cuda.empty_cache()`
5. Use CPU offloading

```
# Example: Load model with CPU offloading
from transformers import AutoModelForCausalLM
```

```
model = AutoModelForCausalLM.from_pretrained(  
    "mistralai/Mistral-7B-v0.1",  
    device_map="auto", # Automatically distribute across GPU/CPU  
    torch_dtype=torch.float16 # Use half precision  
)
```

8.2 Conda/Mamba Environment Issues

Problem: Package conflicts or corrupted environment

Solutions:

```
# Clean conda cache  
mamba clean --all  
  
# Remove and recreate environment  
mamba env remove -n local-ai  
mamba create -n local-ai python=3.10 -y  
  
# Update mamba  
mamba update mamba
```

8.3 Docker Permission Issues

Problem: permission denied while trying to connect to Docker daemon

Solution:

```
# Add user to docker group  
sudo usermod -aG docker $USER  
  
# Log out and back in, or run:  
newgrp docker  
  
# Verify  
docker run hello-world
```

8.4 GPU Not Detected

Checklist:

1. Run `nvidia-smi` - Should show GPU
2. Check driver: `nvidia-smi | grep "Driver"`
3. Verify CUDA: `nvcc --version`
4. Check PyTorch: `python -c "import torch; print(torch.cuda.is_available())"`
5. Update NVIDIA driver if needed

Part 9: Next Steps and Resources

9.1 What's Next?

You now have a complete local AI environment! In the next notebooks, we will:

1. **Notebook 2:** Run local LLMs and understand model architectures
2. **Notebook 3:** Implement semantic search with embeddings and FAISS
3. **Notebook 4:** Work with biological sequences (DNA, proteins)
4. **Notebook 5:** Build a RAG (Retrieval-Augmented Generation) system
5. **Notebook 6:** Fine-tune models on biological data

9.2 Additional Resources

Documentation:

- PyTorch: <https://pytorch.org/docs/>
- Hugging Face Transformers: <https://huggingface.co/docs/transformers/>
- llama.cpp: <https://github.com/ggerganov/llama.cpp>
- Ollama: <https://ollama.com/>
- Conda: <https://docs.conda.io/>

Learning:

- Hugging Face Course: <https://huggingface.co/learn/nlp-course/>
- FastAI: <https://course.fast.ai/>
- Deep Learning Book: <https://www.deeplearningbook.org/>

Community:

- Hugging Face Forums: <https://discuss.huggingface.co/>
- PyTorch Forums: <https://discuss.pytorch.org/>
- r/LocalLLaMA: <https://reddit.com/r/LocalLLaMA>

9.3 Exercises

Exercise 1: Environment Creation Create a new conda environment called "bio-analysis" with Python 3.10, biopython, pandas, and matplotlib.

Exercise 2: GPU Benchmark Write a Python script that compares matrix multiplication speed on CPU vs GPU.

Exercise 3: Model Download Use Hugging Face to download a small model (e.g., "distilbert-base-uncased") and inspect its configuration.

Exercise 4: Docker Container Create a Dockerfile that installs PyTorch, Transformers, and runs a simple script.

Conclusion

Congratulations! You have successfully: ✓ Installed Conda/Mamba for environment management ✓ Set up Docker and NVIDIA Container Toolkit ✓ Installed CUDA and PyTorch with GPU support ✓ Installed multiple LLM frameworks (llama.cpp, Ollama, Transformers) ✓ Tested your setup with real models ✓ Learned troubleshooting techniques

You are now ready to run sophisticated AI models locally for your bioinformatics research!

Important Notes for Biology Researchers:

- Always check data privacy requirements before choosing local vs cloud
- Start with smaller models (7B parameters) before scaling up
- Use quantized models (Q4, Q8) to save memory
- Keep your NVIDIA drivers updated for best performance
- Back up your conda environments regularly

In the next notebook, we'll dive into running LLMs locally and understanding different model architectures!